

Gen-ai

~ Training Program

Name : Mohammed Afnan

Email : afnanmohd5578@gmail.com

Ph. No. : 8790112677

**College : Chaitanya Bharathi Institute
of Technology**

Assignment - 1: Research on ChromaDB

1. Architecture Overview

ChromaDB is an open-source, AI-native vector database designed as the "long-term memory" layer for LLMs.

Data Hierarchy Model

- **Tenants:** Highest logical tier; provides multi-tenant isolation, access control, and quota enforcement.
- **Databases:** Namespaces within a tenant to isolate specific applications.
- **Collections:** Equivalent to relational tables. A schema-less group containing:
 - **Documents:** Raw text chunks.
 - **Embeddings:** High-dimensional vectors.
 - **Metadata:** Key-value pairs for filtering.
 - **IDs:** Unique string keys.

Storage & Computation Backends

- **Single-Node (Local):** Embedded execution via `PersistentClient` or `EphemeralClient`. Uses **SQLite** for metadata and local storage.
- **Distributed Mode:** Decouples compute from storage. Uses **Object Storage** (e.g., AWS S3) for vector indices and a Write-Ahead Log (WAL) for real-time sync.

- **Vector Engine:** Employs Hierarchical Navigable Small World (HNSW) graphs for fast Approximate Nearest Neighbour (ANN) search.

2. Core Features

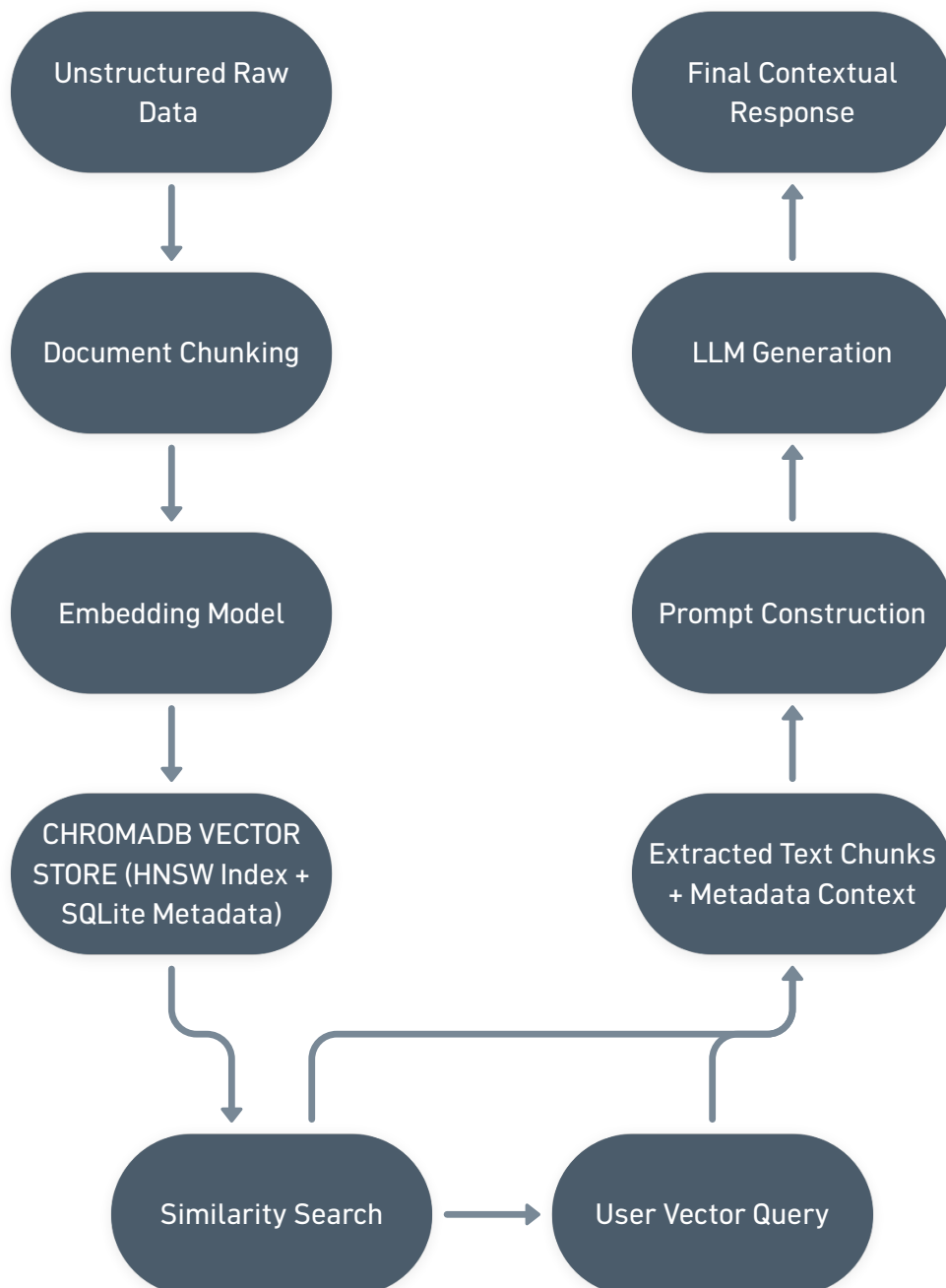
- **Native Embedding Abstraction:** Automatically converts raw text to vectors using a built-in `all-MiniLM-L6-v2` model, with native support for OpenAI, Cohere, and Gemini APIs.
- **Hybrid Search:** Combines dense semantic vector search with sparse keyword search (BM25), merging results via Reciprocal Rank Fusion (RRF).
- **Metadata Filtering:** Supports execution-stage logical filtering via `where` (metadata) and `where_document` (content patterns) clauses.
- **SDK Ecosystem:** Schema-less architecture with official clients for Python and TypeScript, plus community wrappers for Go, Rust, and Java.

3. Industry Use Cases

- **Retrieval-Augmented Generation (RAG):** Context provider for LLMs; fetches relevant document embeddings to minimise model hallucinations.
- **Conversational Memory:** Stores sequential chat logs, enabling autonomous AI agents to retain context across multi-turn sessions.
- **Semantic Recommendations:** Maps user profiles and items into a shared vector space to deliver context-based recommendations rather than exact keyword matches.

Assignment - 2 : Design Memory Flow Diagram

15/06/26, 11:47 AM



Assignment - 3 : Traditional Search vs Semantic Search

Architectural Feature	Traditional Search (Lexical / Keyword)	Semantic Search (Vector Space)
Core Mechanism	Maps exact character sequences using indices like inverted tables (e.g., TF-IDF, BM25).	Translates strings into high-dimensional geometric coordinate vectors using Deep Learning encoders.
Matching Rule	String matching. If words don't match exactly, the document is missed.	Concept and contextual proximity matching using distance formulas like Cosine Similarity or Euclidean Distance.
Handling of Synonyms	Requires manual synonym mapping, lemmatization, or hardcoded alias dictionaries.	Automatically recognizes that terms like " <i>automobile</i> " and " <i>car</i> " occupy similar coordinate space positions.
Context Awareness	Treats words as isolated tokens; ignores structural intent and polysemy.	Evaluates the sentence layout, grammatical dependencies, and underlying user intent.
Computational Footprint	High throughput, fast index generation, and lightweight CPU/RAM usage.	Highly intensive; demands GPU/TPU operations for inference and massive RAM for loading HNSW index graphs.
Query Example	Searching " <i>hot weather drinks</i> " returns documents explicitly containing those exact tokens.	Searching " <i>hot weather drinks</i> " can surface content on " <i>iced matcha</i> " or " <i>cold refreshments</i> " without explicit keyword overlaps.